[WORKSHOP / CTF DIFENSIVO / 4H30]

Blind Trust
Caccia alle vulnerabilità

Quando il codice generato da IA sembra giusto,
ma il processo non lo ha ancora verificato.



AppSec • Supply chain • Secure coding • DevSecOps

[SYS_CHECK / APERTURA]

Se il codice sembra corretto, quali prove servono per fidarsene?



La fiducia non è un controllo: è un'ipotesi da mettere alla prova.

[CONTESTO / VERIFY, DO NOT TRUST]

Il problema non è l'IA. È l'accettazione senza evidenza.

Fiducia implicita

- Un suggerimento plausibile entra nel repository.
- Un commento rassicurante sostituisce l'analisi.
- La pipeline controlla stile, non rischio.

Responsabilità verificabile

- Verificare origine, flusso e privilegi.
- Testare il comportamento, non il tono del commento.
- Rendere i controlli ripetibili in CI.

[OUTCOMES / 6 COMPETENZE]

Cosa porterete a casa

Dipendenze
verificabili e tracciabili

Flussi dati
da source a sink

Segreti
trattati come compromessi

Authorization
server-side e testata

Quality gate
SAST, SCA, secret scan

Review LLM
basata sul rischio

[METODO / CTF DIFENSIVO]

Non una gara a sfruttare: una gara a verificare e mitigare.



[SCENARIO / ASTRASHOP]

Rilascio pilota tra una settimana.

AstraShop è una piattaforma B2B per catalogo, ticket e ordini.
Il team ha usato un LLM per endpoint, utility, test e configurazioni CI.
L'audit interno ha trovato segnali che non possono arrivare in produzione.

[AUDIT SIGNALS]

- Dipendenza fuori catalogo approvato
- Campo “sanitizzato” in report HTML
- Chiave demo in note, log e history
- Utente autenticato, risorsa non propria
- Pipeline: stile sì, security no

[MISSION / SECURITY ENABLEMENT]

Il vostro ruolo: fermare il rischio prima del go-live.



Security Enablement Team

Leggete il repository.
Contestualizzate i finding.
Aiutate il team a rilasciare in sicurezza.

[MISSION OUTPUT]

Raccomandazione
go / no-go condizionata

- ✓ Finding con evidenza riproducibile
- ✓ Mitigazioni ordinate per priorità
- ✓ Controlli che impediscano la ricorrenza

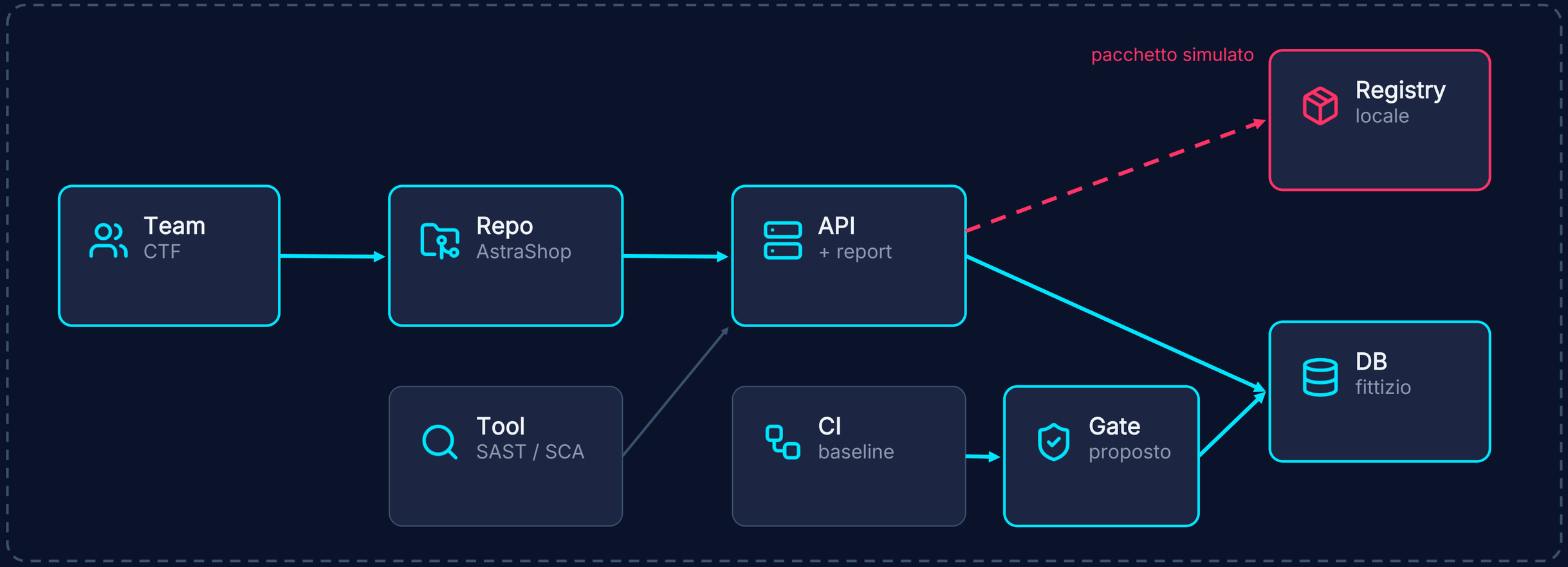
[SAFE BY DESIGN / LAB ONLY]

Un laboratorio sicuro per imparare da vulnerabilità reali.



[ARCHITECTURE / CTF_SCOPE ONLY]

Mappa del laboratorio



[ROE / SAFE PRACTICE]

Rules of Engagement

Operate con curiosità tecnica, ma solo dentro l'ambiente assegnato.

01 Scope chiuso: solo CTF assegnato

02 Nessun target, dominio o servizio esterno

03 Mai dati, password o token reali

04 Nessuna persistenza fuori dal lab

05 PoC minime, innocue e documentate

06 Nessuna esfiltrazione di dati

07 Segnala, poi passa alla mitigazione

08 Chiedi al facilitatore se sei incerto

09 Collaborazione leale tra team

10 Teardown e pulizia al termine

[STOP] Se un'azione oltrepassa il perimetro: fermati e chiedi.

[MISSION FLOW / EVIDENCE FIRST]

Come si completa una flag



Non basta trovare il problema: serve renderlo meno probabile domani.

[FLAG_1 / SUPPLY CHAIN]

Il pacchetto che suona familiare

Slopsquatting simulato: un nome plausibile non è una dipendenza attendibile.

[COMMENTO GENERATO]

“Usa @astrashop/text-normalizer per normalizzare in sicurezza le note dei ticket.”



[RED FLAGS]

Owner generico • repository assente • scope non approvato
Lockfile e integrità da verificare

Consegna: 3 segnali, una patch, una policy di approvazione dipendenze.

[FLAG_1 / CHECK EVIDENZE]

Prima di installare, verificare.

Nome e scope
esistono nel catalogo approvato?

Owner e sorgente
repo, manutenzione, licenza

Registry
origine consentita e scope privato

Lockfile
versione e URL risolto

Integrità
hash e build riproducibile

Approvazione
review e SBOM aggiornate

Se una prova critica manca, **la dipendenza non entra** nel build.

[FLAG_2 / INPUT HANDLING]

Sanitized ≠ Safe

Una funzione rassicurante non protegge un dato nel contesto di output sbagliato.

[FALSO MITO]

“C’è sanitizeNote(), quindi è sicuro.”

Blacklist generica, nel punto sbagliato, non equivale a una barriera al sink.

vs

[REALTÀ OPERATIVA]

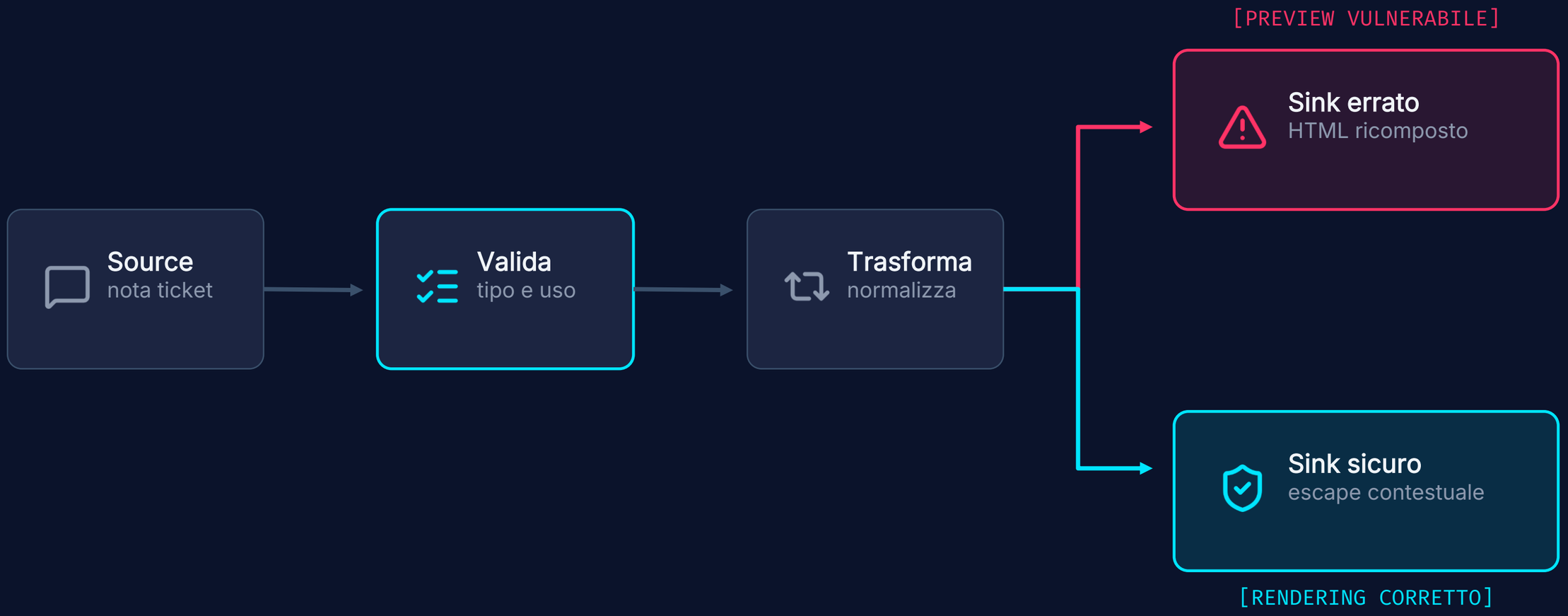
Conta il flusso reale del dato.

Validazione semantica + API sicura + encoding contestuale al momento dell'output.

L'obiettivo della flag è spiegare dove il controllo fallisce e come correggerlo.

[FLAG_2 / SOURCE → SINK]

Segui il dato, non il nome della funzione.



Validare non è fare encoding. L'encoding si applica nel **contesto di output**.

[FLAG_3 / SECRET EXPOSURE]

Il segreto nel contesto

docs/llm-development-notes.md

```
## Debug locale
Usa questa chiave per testare:
TRAINING_ONLY_API_KEY=
demo_key_not_real
Nota: rimossa dal file .env
```

Nel lab è fittizia. Nel mondo reale: trattala come compromessa.

[DOVE CERCARE]

Note • log • fixture • history Git • artifact CI

[RISPOSTA ALL'INCIDENTE]

1. Ruota / revoca → 2. Contieni → 3. Rimuovi
4. Scansiona → 5. Previeni

Un segreto non si “nasconde”: si ruota e si corregge il processo che lo ha esposto.

[FLAG_4 / BOLA - IDOR]

Autenticato ≠ autorizzato

Autenticazione

Chi sei? Sessione o token validi.
Non dice nulla sull'oggetto richiesto.

Autorizzazione

Puoi farlo su questa risorsa?
Owner, ruolo, tenant e azione devono combaciare.

✗
```
findOrderById(orderId)
```

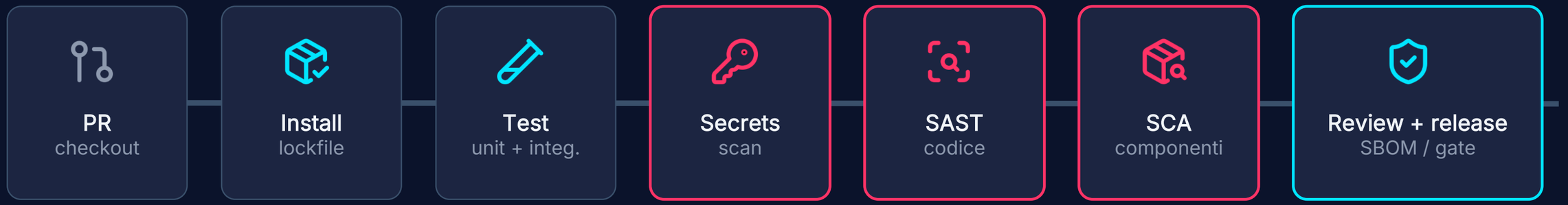
✓
```
findOrder({ id: orderId, tenantId:
currentTenant })
```

La UI può nascondere un link. Solo il server può far rispettare ownership e tenant.

[FLAG_5 / DEVSECOPS GATES]

La pipeline non può fidarsi del codice.

Dal finding manuale a controlli automatici, proporzionati e ripetibili.



[BLOCK] segreto ad alta confidenza • CVE critica • dipendenza non approvata

Gli scanner restringono il rischio. La review umana mantiene la responsabilità.

[SCORING / QUALITY OVER SPEED]

Punteggio, hint e qualità

Il CTF premia la qualità della decisione, non la spettacolarità dell'exploit.

420	565	50
Evidenza	**Mitigazione**	**Debrief**
finding tecnici motivati	patch, test, policy e gate	trade-off e priorità

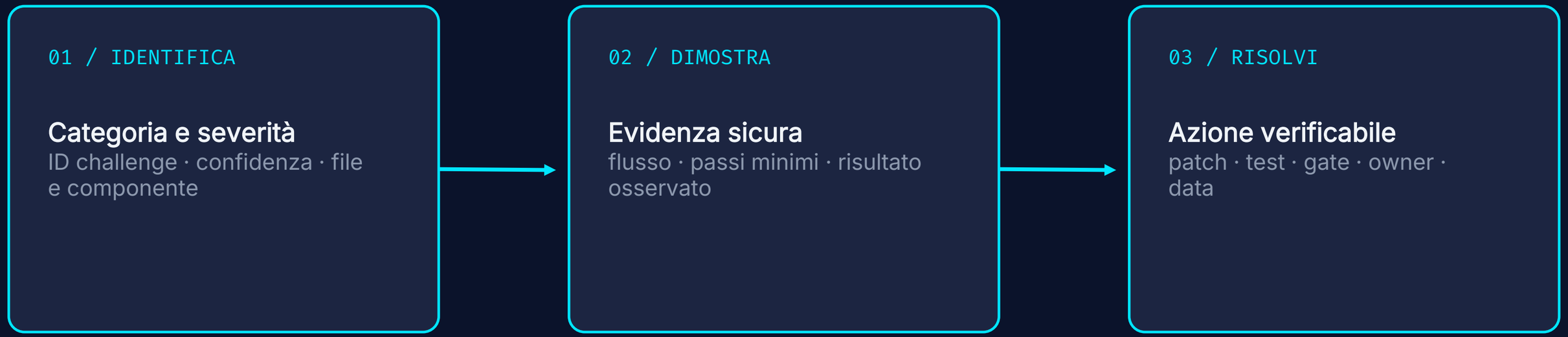
Totale: 1.035

Hint 1 gratuito dopo 10 min • Hint 2: -15% • Soluzione guidata: -30%

[DELIVERABLE / FINDING CARD]

Il deliverable di ogni team

Un finding utile trasforma un problema tecnico in una decisione pronta per il team.



Un buon finding spiega **perché** conta, non soltanto quale riga cambiare.

[PR CHECKLIST / LLM-ASSISTED CODE]

Checklist di review per output LLM

Dipendenze
Esiste?
Owner verificato?
Registry approvata?
Lockfile aggiornato?

Flussi
Input non fidati?
Tipo e semantica?
Sink corretto?
Encoding contestuale?

Authorization
Ownership lato server?
Tenant coerente?
Ruolo e azione?
Test cross-user?

Segreti
Niente nei prompt?
Niente nei log?
Placeholder chiari?
Scan attivo?

Prove
Test negativi?
SAST?
SCA?
Review indipendente?

Una review efficace chiede: “Quale assunzione stiamo accettando senza prova?”

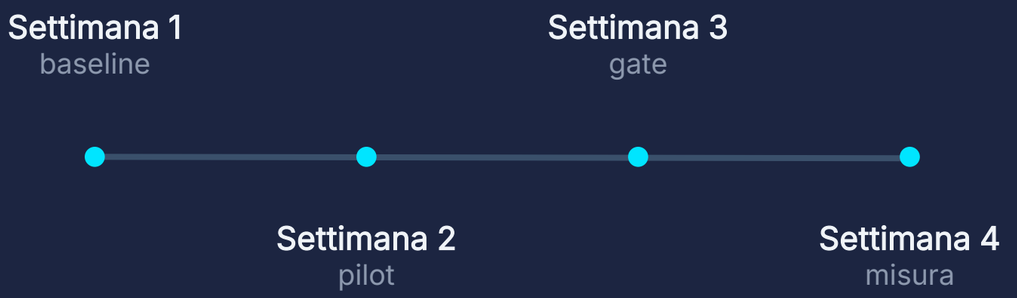
[ACTION PLAN / 30 DAYS]

Dal laboratorio al vostro SDLC

[DEFINITION OF DONE]

- ✓ Codice compreso da un reviewer umano
- ✓ Nuove dipendenze verificate e approvate
- ✓ Test negativi e di authorization, dove rilevanti
- ✓ Nessun segreto o contesto sensibile
- ✓ Secret scan, SAST e SCA conformi alla policy
- ✓ Review indipendente e eccezioni con scadenza

[PRIMI 30 GIORNI]



Partire piccolo, misurare, correggere, estendere.

Verify, do not trust.

L'IA accelera la produzione di codice.
La verifica resta una responsabilità umana e di sistema.

- Verifica ogni **dipendenza**
- Traccia ogni **source → sink**
- Rendi la pipeline **un guardrail**

Riferimenti: OWASP GenAI Security Project • NIST Secure Software Development Framework